

CloudSync — 3-Day Build Plan

EduRankAI Software Development Evaluation · Focused Linux/Rust Systems MVP

Objective: Build a complete, defensible synchronization utility in three days. The goal is not a Dropbox clone; it is a working end-to-end system with a clear engineering thesis, measurable validation, and honest documentation.

Frozen MVP Scope

1	Remote authentication / connection	CLI login and connection to a simple remote server; no Google Drive/OneDrive integration.
2	Directory registration	Register a local directory as a sync target and associate it with a remote location.
3	Filesystem watcher	Event-driven monitoring of create/modify/delete/rename changes; no wasteful polling loop.
4	Manifest builder	Track relative path, size, modified time, and chunk information.
5	Chunking / hashing engine	Fixed-size chunks for MVP; compare hashes and transfer only required chunks.
6	Upload / download pipeline	Transfer, verify integrity, commit manifest safely, and recover from interrupted transfers.

Architecture to Keep Stable

Filesystem → Watcher → Current Manifest → Sync Planner → Remote Manifest → Chunk Engine → Transfer → Verify → Commit Manifest

State separation: the watcher reports local changes; the sync engine decides what to do. Keep current local state, last-synced state, and remote state conceptually separate so a cloud-originated download does not accidentally become an upload loop.

Day 1 — Explore, Decide, Design, Start

Block	Work	Done when
1	Opportunity + workflow	Write the real problem: personal asynchronous replication between Linux machines through a server.
2	Landscape	Briefly compare Drive/OneDrive-style sync, rsync-like approaches, and other relevant tools. Identify
3	Alternatives	Document at least 3 possible project directions and why they were rejected/selected. CloudSync be
4	Architecture	Define CLI, watcher, manifest, sync planner, chunk engine, remote storage and transfer boundaries
5	Data model	Freeze manifest fields, remote layout, chunk identity, file ordering and synchronization state.
6	Foundation	Rust project builds; CLI skeleton works; authentication/connection and directory registration are sta
7	Evidence	Capture architecture diagram, decision notes, initial repository state and Day-1 progress.

Day-1 exit criterion: you have a running project skeleton and a frozen architecture. Do not spend Day 1 polishing the CLI.

Day 2 — Build the Complete Happy Path

Block	Work	Done when
1	Watcher	Filesystem events update local state immediately; sync is not triggered for every event.
2	Manifest	Directory can be represented deterministically and compared with remote state.
3	Chunk engine	Files split into fixed-size chunks; hashes are generated and compared.
4	Planner	Determine upload/download/no-op actions from local and remote state.
5	Upload	New/changed chunks reach the server and are verified.
6	Download	Remote-newer files can be reconstructed locally and verified.
7	Commit	Manifest is updated only after successful transfer/verification.

Block	Work	Done when
8	CLI integration	add/sync/watch/status (and dry-run if time permits) operate through the same pipeline.

Day-2 exit criterion: Machine A → Server → Machine B works for the core file workflow. If this is not true yet, stop adding features and finish the pipeline.

Day 3 — Break It, Improve It, Prove It

Test	Procedure	Evidence
Initial sync	Upload a reasonably large file.	Manifest + transfer output + resulting remote state.
Incremental change	Modify a small region of the file.	Show changed chunks transferred rather than the entire file.
No-op	Run sync without changes.	Show zero unnecessary transfer.
Download	Make the remote copy newer and sync another machine.	Downloaded file + integrity check.
Interrupted transfer	Kill a transfer part-way through.	Restart and demonstrate recovery/resumption.
Watcher	Create/modify/delete/rename files.	Event log / resulting manifest state.
Failure / limitation	Intentionally find one meaningful weakness.	Document failure, cause, attempted fix, and before/after result.

Chunking Decision — Defensible Trade-off

MVP: fixed-size chunks. They are simple to implement, easy to reason about, and make incremental transfer measurable. A middle insertion can shift later fixed chunks and therefore cause redundant transfer.

Alternative considered: content-defined chunking. It can preserve chunk boundaries better across insertions, but adds chunk-boundary computation and implementation complexity. For a personal asynchronous workload, the MVP chooses simplicity and predictable behavior over maximum deduplication efficiency.

Future/maintenance idea: periodically re-chunk large files to establish a clean fixed-size baseline. Do not implement this unless the core system is already complete. It is a documented trade-off, not a reason to miss the deadline.

Documentation — Build It Alongside the Code

Artifact	What to record
Opportunity report	Problem, users/workflow, landscape, alternatives, selection and thesis.
Engineering notes	Architecture, manifest design, chunk strategy, remote storage, sync semantics and trade-offs.
Validation log	Tests, commands, outputs, measurements, failures and fixes.
README	Problem, architecture, setup, server requirements, CLI usage, limitations and examples.
Final report	Executive summary → opportunity → exploration → alternatives → thesis/scope → engineering → implementation → validation
Declaration	AI assistance, libraries/frameworks/APIs/docs and any external assets/tools used. Be completely honest.

Hard Scope Rules

- No GUI.
- No provider-specific cloud integration in the MVP.
- No OAuth unless the selected server actually requires it.
- No microservices, database, dashboard, Kubernetes or cloud theatre.
- No content-defined chunking unless the core MVP is already complete.
- No full conflict-resolution system; document the timestamp-based asynchronous-use assumption.
- If a feature does not strengthen the six core components, it waits.
- When the happy path works, spend remaining time on failure handling, tests, evidence and documentation.

Submission Checklist

- 01_Final_Report.pdf
- 02_Codebase/README.md + source
- 03_Evidence/
- 04_Demo/
- 05_Declaration/
- Clean build from a fresh checkout
- Core A → Server → B workflow demonstrated
- Incremental chunk transfer demonstrated
- At least one failure/weakness + improvement documented
- All AI/external assistance and libraries declared honestly

Final rule: A small system that works end-to-end, has evidence, and that you can explain line-by-line is worth far more than a sophisticated system with unfinished pieces. Freeze the scope and ship.